

**UNIVERSIDADE ESTADUAL DE MATO GROSSO DO SUL UNIDADE
UNIVERSITÁRIA DE DOURADOS**

**CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA DE
PROGRAMAÇÃO PARALELA**

IGOR MONTEIRO NUNES

**ÁRVORE GERADORA MÍNIMA COM EXECUÇÃO SEQUENCIAL E
PARALELA
UTILIZANDO MPI**

**Dourados,
MS 2026**

IGOR MONTEIRO NUNES

**ÁRVORE GERADORA MÍNIMA COM EXECUÇÃO SEQUENCIAL E
PARALELA
UTILIZANDO MPI**

Trabalho acadêmico do curso de Ciência da Computação apresentado à disciplina de Programação Paralela da Universidade Estadual de Mato Grosso do Sul, Unidade Universitária de Dourados, como exigência para composição da terceira nota.

Prof. Dr. Rubens Barbosa Filho

**Dourados,
MS 2026**

Sumário

1	Sumário do problema	3
2	Algoritmo sequencial	4
2.1	Estruturas e funções principais	4
2.2	Leitura em blocos	5
2.3	Funcionamento geral	5
2.4	Medição de tempo e saída	6
3	Algoritmo paralelo	7
3.1	Inicialização e divisão das arestas	7
3.2	Estruturas principais	7
3.3	Distribuição e comunicação inicial	8
3.4	Busca local e redução global	9
3.5	Sincronização das uniões	10
3.6	Compactação e medição de tempo	10
4	Decisões de implementação	12
5	Comunicação entre as máquinas	14
5.1	Distribuição das arestas	14
5.2	Tipos MPI personalizados	14
5.3	Redução e sincronização das uniões	15
5.4	Resumo da comunicação	15
6	Ambiente de execução	16
7	Testes e resultados	17
8	Gráfico comparativo de tempo	18

9	Gráfico dos tempos paralelos	19
10	Speedup	20
11	Eficiência	21
12	Lei de Amdahl	22
13	Logs de execução	24
14	Análise dos resultados	25
15	Conclusão e Referências Bibliográficas	26

1 Sumário do problema

O objetivo deste trabalho é implementar e comparar uma solução sequencial e uma solução paralela para o problema da Árvore Geradora Mínima de um grafo ponderado.

A Árvore Geradora Mínima corresponde a um conjunto de arestas que conecta todos os vértices do grafo com o menor peso total possível, sem formar ciclos. Para resolver esse problema, foi utilizado o algoritmo de Borůvka em conjunto com a estrutura Union-Find.

O grafo utilizado nos testes possui grande volume de dados, contendo:

- 10.000.000 de vértices;
- 800.000.000 de arestas;
- arquivo de entrada em formato binário;
- cada aresta composta por origem, destino e peso.

Foram desenvolvidas duas versões do programa: uma versão sequencial, utilizada como base de comparação, e uma versão paralela utilizando MPI, executada com 4, 8, 12 e 16 processos.

2 Algoritmo sequencial

A versão sequencial foi implementada em linguagem C e utiliza o algoritmo de Boruvka para encontrar a Árvore Geradora Mínima. Esse algoritmo funciona em rodadas: em cada rodada, cada componente do grafo escolhe sua menor aresta de saída. As arestas escolhidas são então avaliadas e adicionadas à AGM apenas quando conectam componentes diferentes.

Para controlar os componentes e evitar ciclos, foi utilizada a estrutura Union-Find. Inicialmente, cada vértice é tratado como um componente separado. Conforme as arestas são adicionadas à AGM, os componentes são unidos. O processo termina quando resta apenas um componente.

Cada aresta do grafo é representada pela estrutura **Aresta**, composta pelo vértice de origem, vértice de destino e peso:

```
1 typedef struct {  
2     int origem;  
3     int destino;  
4     double peso;  
5 } Aresta;
```

O grafo utilizado possui 10.000.000 de vértices e 800.000.000 de arestas, armazenadas em arquivo binário nesse formato.

2.1 Estruturas e funções principais

As principais estruturas auxiliares utilizadas na versão sequencial foram:

- **raiz**: armazena o representante de cada vértice no Union-Find;
- **rank**: auxilia na união dos componentes;
- **menor**: guarda a menor aresta encontrada para cada componente;
- **agm**: armazena as arestas escolhidas para formar a Árvore Geradora Mínima;
- **buffer**: usado para ler o arquivo de entrada em blocos.

A estrutura Union-Find é formada principalmente pelas funções **find** e **unionSet**. A função **find** encontra o representante de um componente usando compressão de caminho, enquanto **unionSet** une dois componentes diferentes usando rank.

```
1 int find(int raiz[], int x) {  
2     if (raiz[x] != x)  
3         raiz[x] = find(raiz, raiz[x]);  
4  
5     return raiz[x];  
6 }
```

Antes de adicionar uma aresta à AGM, o programa verifica se seus vértices pertencem a componentes diferentes. Essa verificação impede a formação de ciclos.

A função `melhor` é usada para escolher a melhor aresta de cada componente. O critério principal é o menor peso. Em caso de empate, são usados os campos `origem` e `destino` como desempate.

```
1 if (a.peso < b.peso)
2     return 1;
3
4 if (a.peso == b.peso) {
5     if (a.origem < b.origem)
6         return 1;
7
8     if (a.origem == b.origem && a.destino < b.destino)
9         return 1;
10 }
```

2.2 Leitura em blocos

Como o arquivo de entrada é muito grande, ele não é carregado completamente na memória. A leitura é feita em blocos de 1.000.000 de arestas, e cada bloco é processado imediatamente após a leitura.

```
1 size_t lidas = fread(buffer, sizeof(Aresta), qtdLer, entrada);
```

Durante o processamento, o programa ignora arestas inválidas, como vértices fora do intervalo permitido, laços, pesos negativos ou pesos iguais a `DBL_MAX`.

```
1 if (u < 0 || u >= V || v < 0 || v >= V)
2     continue;
3
4 if (u == v)
5     continue;
6
7 if (peso < 0.0 || peso == DBL_MAX)
8     continue;
```

Essa validação evita que dados incorretos interfiram na construção da AGM.

2.3 Funcionamento geral

O laço principal executa enquanto existir mais de um componente no grafo. Em cada rodada, o vetor `menor` é reinicializado e o arquivo é percorrido em blocos. Para cada aresta válida, o programa identifica os componentes dos vértices de origem e destino.

```
1 int compU = find(raiz, u);
2 int compV = find(raiz, v);
```

Se os vértices pertencem a componentes diferentes, a aresta pode atualizar a menor aresta de saída dos dois componentes envolvidos.

```
1 if (compU != compV) {
2     if (melhor(atual, menor[compU]))
3         menor[compU] = atual;
4
5     if (melhor(atual, menor[compV]))
6         menor[compV] = atual;
7 }
```

Após a varredura das arestas, o programa percorre o vetor **menor**. As arestas selecionadas são adicionadas à AGM somente se ainda conectarem componentes diferentes.

```
1 if (compU != compV) {
2     agm[qtdAGM++] = menor[i];
3
4     unionSet(raiz, rank, compU, compV);
5
6     componentes--;
7 }
```

Essa segunda verificação é necessária porque, dentro de uma mesma rodada, alguns componentes podem já ter sido unidos anteriormente.

2.4 Medição de tempo e saída

A medição de tempo começa antes do laço principal e termina logo após a construção da AGM.

```
1 clock_gettime(CLOCK_MONOTONIC, &inicio);
2
3 while (componentes > 1) {
4     ...
5 }
6
7 clock_gettime(CLOCK_MONOTONIC, &fimAlgoritmo);
```

A ordenação final, a soma do peso total e a escrita do arquivo de saída não entram no tempo medido. Essas etapas são tratadas como pós-processamento, para manter o mesmo critério usado na versão paralela.

Após o término do algoritmo, as arestas da AGM são ordenadas com **qsort** apenas para padronizar a saída e a soma do peso total. Ao final, o programa gera o arquivo **saida_seq.txt** e exibe no terminal o peso total da AGM, o tempo medido e o nome do arquivo gerado.

3 Algoritmo paralelo

A versão paralela foi implementada em linguagem C utilizando MPI. Assim como na versão sequencial, foi utilizado o algoritmo de Boruvka com Union-Find. A principal diferença é que, no paralelo, as arestas são divididas entre os processos, permitindo que cada rank analise apenas uma parte do grafo.

O rank 0 é responsável por abrir o arquivo de entrada, dividir as arestas e enviá-las aos demais processos. Depois da distribuição, cada processo trabalha com seu vetor de arestas locais. Em cada rodada, os processos escolhem localmente as menores arestas de saída dos componentes. Essas escolhas são combinadas no rank 0, que decide quais arestas entram na AGM. As uniões realizadas são então enviadas para todos os processos, mantendo a estrutura Union-Find sincronizada.

3.1 Inicialização e divisão das arestas

O programa inicia o ambiente MPI e identifica o rank de cada processo e a quantidade total de processos em execução.

```
1 MPI_Init(&argc, &argv);
2 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
3 MPI_Comm_size(MPI_COMM_WORLD, &size);
```

A divisão das arestas é feita com base no rank e no número total de processos. Assim, cada processo recebe uma faixa aproximadamente igual do arquivo de entrada.

```
1 long long inicioLocal = (long long)rank * E / size;
2 long long fimLocal = (long long)(rank + 1) * E / size;
3 long long nLocais = fimLocal - inicioLocal;
```

Com isso, as 800.000.000 de arestas são distribuídas entre os processos participantes da execução.

3.2 Estruturas principais

A estrutura `Aresta` representa as arestas do grafo, enquanto a estrutura `Uniao` armazena os pares de componentes unidos pelo rank 0 em cada rodada.

```
1 typedef struct {
2     int origem;
3     int destino;
4     double peso;
5 } Aresta;
6
7 typedef struct {
8     int u;
9     int v;
```

```
10 } Uniao;
```

As principais estruturas auxiliares usadas na versão paralela foram:

- **arestasLocais**: armazena as arestas recebidas por cada rank;
- **menorLocal**: guarda as melhores arestas encontradas localmente;
- **menorGlobal**: usado pelo rank 0 após a redução global;
- **raiz e altura**: usados pelo Union-Find;
- **componentesAtivos**: guarda os componentes ainda existentes;
- **mapaComponente**: associa cada componente ativo a uma posição compacta;
- **unioes**: armazena as uniões que devem ser aplicadas pelos demais ranks.

Como o MPI não conhece diretamente as estruturas criadas no programa, foram definidos tipos MPI personalizados para enviar **Aresta** e **Uniao** entre os processos.

```
1 MPI_Type_create_struct(3, blocosAresta, deslocamentosAresta,
2                       tiposAresta, &MPI_ARESTA);
3 MPI_Type_commit(&MPI_ARESTA);
4
5 MPI_Type_create_struct(2, blocosUniao, deslocamentosUniao,
6                       tiposUniao, &MPI_UNIAO);
7 MPI_Type_commit(&MPI_UNIAO);
```

O tipo **MPI_ARESTA** é utilizado na distribuição das arestas. O tipo **MPI_UNIAO** é usado para enviar as uniões realizadas pelo rank 0.

3.3 Distribuição e comunicação inicial

A leitura do arquivo é centralizada no rank 0. Ele usa **fseeko** para acessar a posição correta do arquivo correspondente a cada processo, lê as arestas em blocos e envia os dados aos demais ranks.

```
1 off_t deslocamento = (off_t)ini * (off_t)sizeof(Aresta);
2 fseeko(entrada, deslocamento, SEEK_SET);
```

Para cada bloco enviado, o rank 0 envia primeiro a quantidade de arestas e depois o bloco com as arestas.

```
1 MPI_Send(&qtd, 1, MPI_INT, r, TAG_QTD, MPI_COMM_WORLD);
2 MPI_Send(buffer, qtd, MPI_ARESTA, r, TAG_DADOS, MPI_COMM_WORLD);
```

Os demais processos recebem os dados com **MPI_Recv** e armazenam as arestas em seus vetores locais.

```

1 MPI_Recv(&qtd, 1, MPI_INT, 0, TAG_QTD,
2         MPI_COMM_WORLD, MPI_STATUS_IGNORE);
3
4 MPI_Recv(&arestasLocais[recebidas], qtd, MPI_ARESTA,
5         0, TAG_DADOS, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

```

Ao final do envio para cada rank, o rank 0 envia uma quantidade igual a zero, indicando que aquele processo já recebeu todas as suas arestas.

3.4 Busca local e redução global

Depois da distribuição, todos os processos executam as rodadas do algoritmo de Boruvka. Em cada rodada, cada rank percorre apenas suas arestas locais e procura a menor aresta de saída para cada componente ativo.

```

1 for (long long i = 0; i < nLocais; i++) {
2     Aresta aresta = arestasLocais[i];
3
4     int raizU = raiz[aresta.origem];
5     int raizV = raiz[aresta.destino];
6
7     if (raizU == raizV) {
8         internas++;
9         continue;
10    }
11
12    int posU = mapaComponente[raizU];
13    int posV = mapaComponente[raizV];
14
15    if (melhor(aresta, menorLocal[posU]))
16        menorLocal[posU] = aresta;
17
18    if (melhor(aresta, menorLocal[posV]))
19        menorLocal[posV] = aresta;
20 }

```

Arestas internas, ou seja, arestas que ligam vértices do mesmo componente, são ignoradas. As demais podem atualizar o vetor `menorLocal`.

Após a busca local, os resultados são combinados no rank 0 por meio de `MPI_Reduce`. Para isso, foi criada uma operação personalizada que compara as arestas e mantém a menor para cada componente.

```

1 MPI_Reduce(menorLocal, rank == 0 ? menorGlobal : NULL,
2           qtdComponentes, MPI_ARESTA, op_menor_aresta,
3           0, MPI_COMM_WORLD);

```

Com o vetor `menorGlobal`, o rank 0 verifica quais arestas conectam componentes diferentes. Essas arestas entram na AGM, e as uniões correspondentes são armazenadas no vetor `unioes`.

```
1 if (raizU != raizV) {
2     agm[qtdAGM] = menorGlobal[i];
3     qtdAGM++;
4
5     unioes[qtdUnioes].u = raizU;
6     unioes[qtdUnioes].v = raizV;
7     qtdUnioes++;
8
9     unionSet(raiz, altura, raizU, raizV);
10 }
```

3.5 Sincronização das uniões

Depois que o rank 0 escolhe as arestas que entram na AGM, ele envia as uniões realizadas para todos os processos. Primeiro é enviada a quantidade de uniões.

```
1 MPI_Bcast(&qtdUnioes, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

Em seguida, caso exista pelo menos uma união, o vetor `unioes` é enviado para todos os ranks.

```
1 if (qtdUnioes > 0)
2     MPI_Bcast(unioes, qtdUnioes, MPI_UNIAO, 0, MPI_COMM_WORLD);
```

Os ranks diferentes de 0 aplicam localmente as mesmas uniões.

```
1 if (rank != 0) {
2     for (int i = 0; i < qtdUnioes; i++)
3         unionSet(raiz, altura, unioes[i].u, unioes[i].v);
4 }
```

Esse processo garante que todos os ranks mantenham a mesma visão dos componentes do grafo durante a execução.

3.6 Compactação e medição de tempo

Durante as rodadas do algoritmo, muitas arestas passam a ser internas, ou seja, passam a ligar vértices que já estão no mesmo componente. Para evitar processamento desnecessário, o programa compacta o vetor de arestas locais quando pelo menos 30% das arestas locais já são internas.

```
1 if (nLocais > 0 &&
2     internas * 100 >= nLocais * LIMAR_COMPACTACAO) {
```

```
3     ...
4 }
```

A medição do tempo paralelo é feita com `MPI_Wtime`. Antes de iniciar e antes de finalizar a medição, são usadas barreiras para sincronizar todos os processos.

```
1 MPI_Barrier(MPI_COMM_WORLD);
2 double inicioTempo = MPI_Wtime();
3
4 ...
5
6 MPI_Barrier(MPI_COMM_WORLD);
7 double fimAlgoritmo = MPI_Wtime();
```

Assim como na versão sequencial, a ordenação, a soma do peso total e a escrita do arquivo não entram no tempo medido.

Ao final, somente o rank 0 ordena as arestas da AGM, calcula o peso total e gera o arquivo `saida_par.txt`. Cada rank também imprime no terminal a máquina onde foi executado e a quantidade de arestas locais recebidas, auxiliando na verificação da distribuição do trabalho.

4 Decisões de implementação

A primeira decisão foi utilizar o algoritmo de Boruvka para encontrar a Árvore Geradora Mínima. Esse algoritmo foi escolhido porque trabalha em rodadas, nas quais cada componente escolhe sua menor aresta de saída. Essa característica facilita a paralelização, pois diferentes processos podem analisar partes distintas do conjunto de arestas.

Também foi utilizada a estrutura Union-Find nas duas versões do programa. Essa estrutura permite verificar de forma eficiente se dois vértices pertencem ao mesmo componente e também permite unir componentes quando uma aresta é adicionada à AGM. Na versão sequencial, a união utiliza o vetor `rank`. Na versão paralela, foi usado o vetor `altura`, com tipo `unsigned char`, reduzindo o consumo de memória em relação a um vetor de inteiros.

```
1 int *raiz = malloc((size_t)V * sizeof(int));
2 unsigned char *altura = calloc((size_t)V, sizeof(unsigned char));
```

Como o grafo possui 800.000.000 de arestas, a leitura do arquivo foi feita em blocos de 1.000.000 de arestas. Essa decisão evita carregar todo o arquivo em memória de uma vez.

```
1 #define BLOCO 1000000
```

Na versão sequencial, o arquivo é percorrido em blocos a cada rodada do algoritmo. Na versão paralela, o rank 0 lê o arquivo em blocos e distribui as partes correspondentes aos demais processos. A divisão das arestas é feita usando o rank do processo e a quantidade total de processos.

```
1 long long inicioLocal = (long long)rank * E / size;
2 long long fimLocal = (long long)(rank + 1) * E / size;
3 long long nLocais = fimLocal - inicioLocal;
```

Outra decisão importante foi criar tipos MPI personalizados para as estruturas `Aresta` e `Uniao`. Isso permitiu enviar essas estruturas diretamente entre os processos, sem precisar transmitir separadamente cada campo.

```
1 MPI_Type_create_struct(3, blocosAresta, deslocamentosAresta,
2                       tiposAresta, &MPI_aresta);
3 MPI_Type_commit(&MPI_aresta);
```

Também foi criada uma operação personalizada para o `MPI_Reduce`. Essa operação compara as arestas encontradas localmente por cada processo e mantém a menor aresta global de cada componente. Essa decisão foi necessária porque as reduções padrão do MPI não sabem comparar diretamente estruturas do tipo `Aresta` usando peso, origem e destino como critérios.

```
1 MPI_Op_create(reducao_menor_aresta, 1, &op_menor_aresta);
```

Na versão paralela, após o rank 0 escolher as arestas que entram na AGM, as uniões realizadas são enviadas para todos os processos usando `MPI_Bcast`. Isso garante que todos os ranks mantenham a mesma visão dos componentes do grafo.

```
1 MPI_Bcast(&qtdUnioes, 1, MPI_INT, 0, MPI_COMM_WORLD);  
2  
3 if (qtdUnioes > 0)  
4     MPI_Bcast(unioes, qtdUnioes, MPI_UNIAO, 0, MPI_COMM_WORLD);
```

Além disso, foi implementada uma compactação das arestas internas na versão paralela. Quando pelo menos 30% das arestas locais passam a ligar vértices do mesmo componente, essas arestas são removidas do vetor local, pois não podem mais fazer parte da AGM.

```
1 #define LIMIAR_COMPACTACAO 30
```

Por fim, a medição de tempo foi feita apenas sobre a parte principal do algoritmo. A ordenação final, a soma do peso total e a escrita do arquivo de saída foram consideradas pós-processamento e não entraram no tempo medido. Essa decisão foi mantida nas duas versões para tornar a comparação entre sequencial e paralelo mais justa.

5 Comunicação entre as máquinas

A comunicação entre as máquinas foi realizada utilizando MPI. Cada processo possui um identificador chamado **rank**, e o processo de **rank 0** ficou responsável pelas principais etapas de coordenação: leitura do arquivo, distribuição das arestas, recebimento das menores arestas globais e envio das uniões aos demais processos.

A comunicação ocorre em três momentos principais: distribuição inicial das arestas, redução das menores arestas locais e sincronização das uniões realizadas durante o algoritmo.

5.1 Distribuição das arestas

No início da execução, cada processo calcula a quantidade de arestas que deve receber com base no seu **rank** e no total de processos.

```
1 long long inicioLocal = (long long)rank * E / size;  
2 long long fimLocal = (long long)(rank + 1) * E / size;  
3 long long nLocais = fimLocal - inicioLocal;
```

O **rank 0** abre o arquivo de entrada, lê as arestas em blocos e envia os dados para os demais ranks. Para cada bloco, primeiro é enviada a quantidade de arestas e depois o bloco de dados.

```
1 MPI_Send(&qtd, 1, MPI_INT, r, TAG_QTD, MPI_COMM_WORLD);  
2 MPI_Send(buffer, qtd, MPI_ARESTA, r, TAG_DADOS, MPI_COMM_WORLD);
```

Os demais processos recebem essas informações usando **MPI_Recv** e armazenam as arestas recebidas no vetor **arestasLocais**.

```
1 MPI_Recv(&qtd, 1, MPI_INT, 0, TAG_QTD,  
2         MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
3  
4 MPI_Recv(&arestasLocais[recebidas], qtd, MPI_ARESTA,  
5         0, TAG_DADOS, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Foram usadas duas tags para diferenciar as mensagens: **TAG_QTD**, para a quantidade de arestas, e **TAG_DADOS**, para o bloco de arestas. Quando o envio para um processo termina, o **rank 0** envia uma quantidade igual a zero, indicando o fim da transmissão.

5.2 Tipos MPI personalizados

Como o programa trabalha com estruturas próprias, foram criados tipos MPI personalizados. O tipo **MPI_ARESTA** permite enviar estruturas do tipo **Aresta**, contendo origem, destino e peso. O tipo **MPI_UNIAO** permite enviar as uniões realizadas pelo **rank 0**.

```
1 MPI_Type_create_struct(3, blocosAresta, deslocamentosAresta,
```



```

2         tiposAresta, &MPI_ARESTA);
3 MPI_Type_commit(&MPI_ARESTA);
4
5 MPI_Type_create_struct(2, blocosUniao, deslocamentosUniao,
6                       tiposUniao, &MPI_UNIAO);
7 MPI_Type_commit(&MPI_UNIAO);

```

Essa decisão facilitou a comunicação, pois permitiu transmitir as estruturas completas diretamente entre os processos.

5.3 Redução e sincronização das uniões

Após a distribuição, cada rank processa suas arestas locais e encontra as menores arestas de saída dos componentes. Esses resultados são combinados no rank 0 por meio de `MPI_Reduce`, usando uma operação personalizada para comparar as arestas.

```

1 MPI_Reduce(menorLocal, rank == 0 ? menorGlobal : NULL,
2           qtdComponentes, MPI_ARESTA, op_menor_aresta,
3           0, MPI_COMM_WORLD);

```

Depois da redução, o rank 0 decide quais arestas entram na AGM e registra as uniões realizadas. Essas uniões são enviadas para todos os processos com `MPI_Bcast`, garantindo que todos mantenham a mesma estrutura Union-Find.

```

1 MPI_Bcast(&qtdUnioes, 1, MPI_INT, 0, MPI_COMM_WORLD);
2
3 if (qtdUnioes > 0)
4     MPI_Bcast(unioes, qtdUnioes, MPI_UNIAO, 0, MPI_COMM_WORLD);

```

Os processos diferentes de 0 aplicam localmente as mesmas uniões recebidas. Dessa forma, todos os ranks continuam com a mesma visão dos componentes do grafo durante as rodadas do algoritmo.

5.4 Resumo da comunicação

Etapa	Função MPI	Objetivo
Distribuição das arestas	<code>MPI_Send</code> / <code>MPI_Recv</code>	Enviar blocos de arestas aos ranks
Redução das menores arestas	<code>MPI_Reduce</code>	Combinar resultados locais no rank 0
Envio das uniões	<code>MPI_Bcast</code>	Sincronizar o Union-Find entre os processos
Sincronização	<code>MPI_Barrier</code>	Alinhar os processos na medição de tempo

Tabela 1: Principais comunicações utilizadas na versão paralela.

Portanto, a comunicação foi organizada de forma centralizada no rank 0 para a distribuição inicial dos dados e para a escolha global das arestas da AGM.

6 Ambiente de execução

As principais características da máquina utilizada na execução sequencial são apresentadas na Tabela 2.

Item	Especificação
Processador	AMD Ryzen 5 PRO 5650G
Clock	0,40 a 4,47 GHz
Núcleos / threads	6 núcleos físicos, 12 threads
Memória RAM	7,1 GiB
Sistema operacional	Ubuntu 24.04.2 LTS
Kernel	6.17.0-23-generic

Tabela 2: Configuração da máquina utilizada na execução sequencial.

A versão sequencial foi compilada com `gcc`, enquanto a versão paralela foi compilada com `mpicc`. As execuções paralelas foram realizadas com 4, 8, 12 e 16 processos, utilizando arquivos de hosts específicos para cada quantidade de máquinas.

O arquivo de entrada foi informado como parâmetro na execução dos programas e continha 10.000.000 de vértices e 800.000.000 de arestas em formato binário.

Os principais comandos utilizados foram:

```
1 make runseq
2 make run4
3 make run8
4 make run12
5 make run16
```

As informações da máquina foram obtidas por comandos do sistema, como:

```
1 lscpu
2 free -h
3 uname -a
4 gcc --version
5 mpicc --version
```

Essas informações foram registradas para contextualizar os tempos obtidos nas execuções sequencial e paralelas.

7 Testes e resultados

Foram realizadas execuções da versão sequencial e da versão paralela com 4, 8, 12 e 16 processos. Em todas as execuções, o peso total da Árvore Geradora Mínima foi o mesmo, indicando que a versão paralela produziu o mesmo resultado da versão sequencial.

Execução	Peso total da AGM	Tempo total
Sequencial	75117.848736822314	450.847467 s
Paralelo com 4 processos	75117.848736822314	150.940320 s
Paralelo com 8 processos	75117.848736822314	245.580198 s
Paralelo com 12 processos	75117.848736822314	139.602443 s
Paralelo com 16 processos	75117.848736822314	290.081397 s

Tabela 3: Resultados obtidos nas execuções sequencial e paralelas.

O peso total obtido em todas as execuções foi **75117.848736822314**. Esse valor foi utilizado como critério de validação, pois mostra que as versões sequencial e paralela chegaram à mesma AGM em termos de peso total.

Em relação ao tempo, todas as execuções paralelas foram mais rápidas que a execução sequencial. O menor tempo foi obtido com 12 processos, com **139.602443 segundos**. A execução com 4 processos também apresentou bom desempenho, com **150.940320 segundos**.

Nas execuções paralelas, as 800.000.000 de arestas foram divididas entre os processos. Assim, com 4 processos cada rank recebeu aproximadamente 200.000.000 de arestas; com 8 processos, 100.000.000; com 12 processos, aproximadamente 66.666.666 ou 66.666.667; e com 16 processos, 50.000.000 de arestas.

8 Gráfico comparativo de tempo

A Figura 1 compara o tempo da execução sequencial com os tempos obtidos nas execuções paralelas. O objetivo desse gráfico é mostrar o ganho obtido ao distribuir o processamento entre múltiplos processos MPI. O tempo sequencial utilizado como referência foi de **450.847467 segundos**.

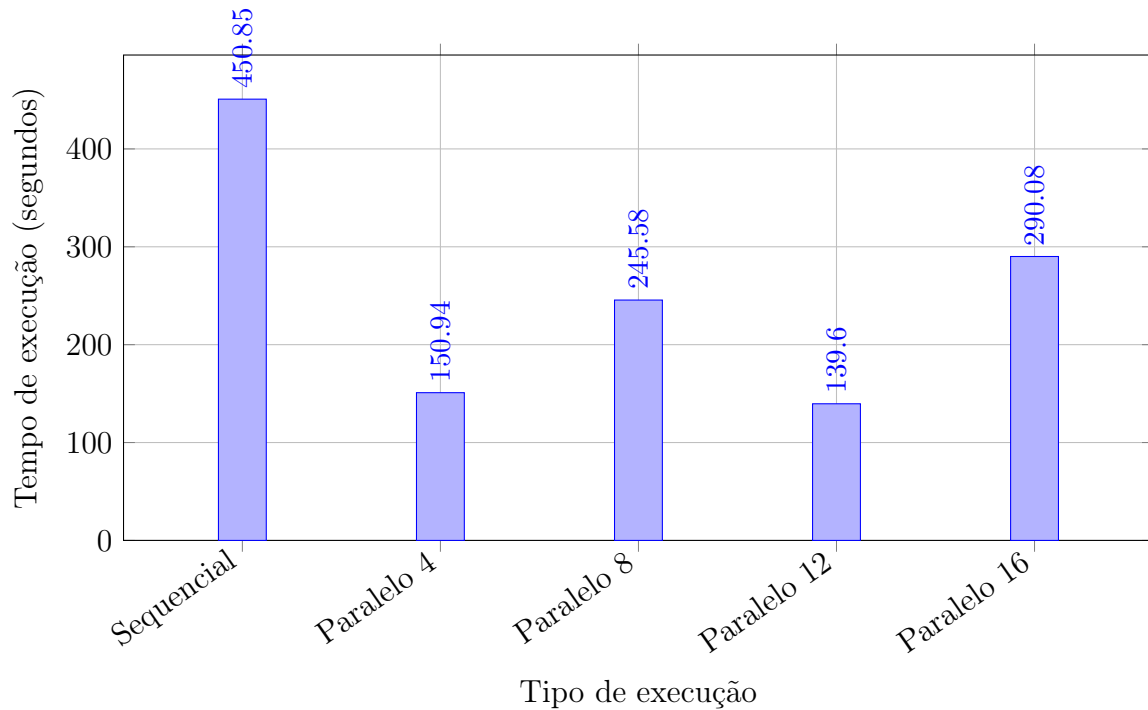


Figura 1: Comparação entre o tempo sequencial e os tempos das execuções paralelas.

Pelo gráfico, é possível observar que a versão paralela reduziu o tempo de execução em relação à versão sequencial em todos os testes realizados. A maior redução ocorreu na execução com **12 processos**, que obteve o tempo de **139.602443 segundos**. Isso mostra que, para esse conjunto de testes, a paralelização conseguiu acelerar a construção da AGM em relação à execução em uma única máquina.

9 Gráfico dos tempos paralelos

A Figura 2 apresenta somente os tempos das execuções paralelas. Diferente do gráfico anterior, aqui o foco é comparar o comportamento da própria versão paralela conforme a quantidade de processos aumenta.

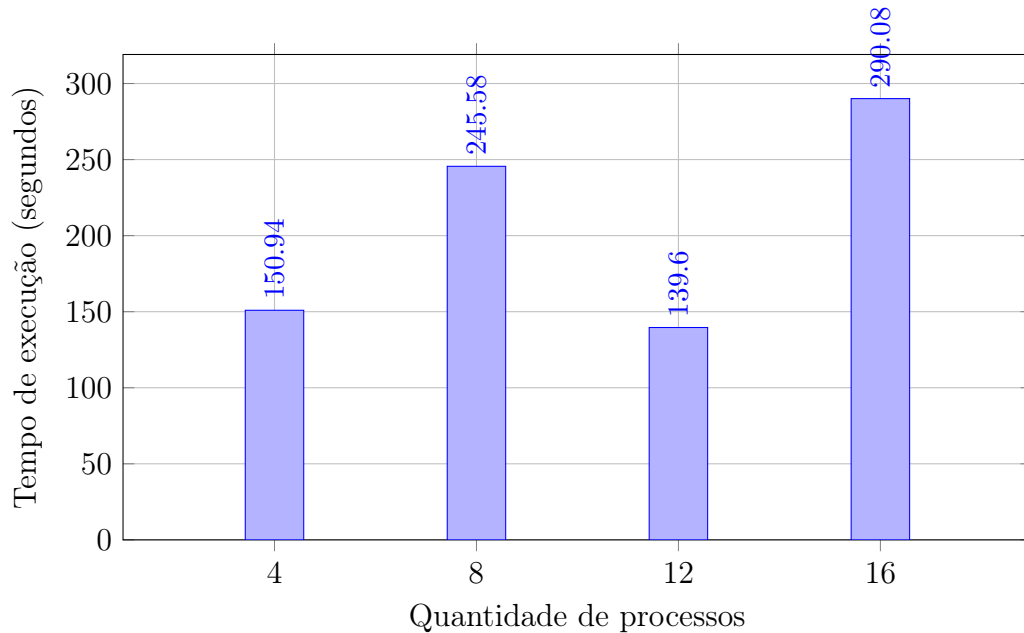


Figura 2: Comparação dos tempos obtidos nas execuções paralelas.

Analisando apenas as execuções paralelas, percebe-se que o melhor tempo não ocorreu com a maior quantidade de processos, mas sim com **12 processos**. A execução com **4 processos** também apresentou resultado próximo, enquanto as execuções com **8** e **16 processos** tiveram tempos superiores.

10 Speedup

O speedup indica quantas vezes a versão paralela foi mais rápida que a versão sequencial. Ele foi calculado pela razão entre o tempo sequencial e o tempo paralelo para cada quantidade de processos:

$$S(p) = \frac{T_{seq}}{T_{par}(p)}$$

Neste trabalho, foi utilizado como referência o tempo sequencial de **450.847467 segundos**.

Processos	Tempo paralelo (s)	Speedup
4	150.940320	2.9869
8	245.580198	1.8358
12	139.602443	3.2295
16	290.081397	1.5542

Tabela 4: Speedup obtido nas execuções paralelas.

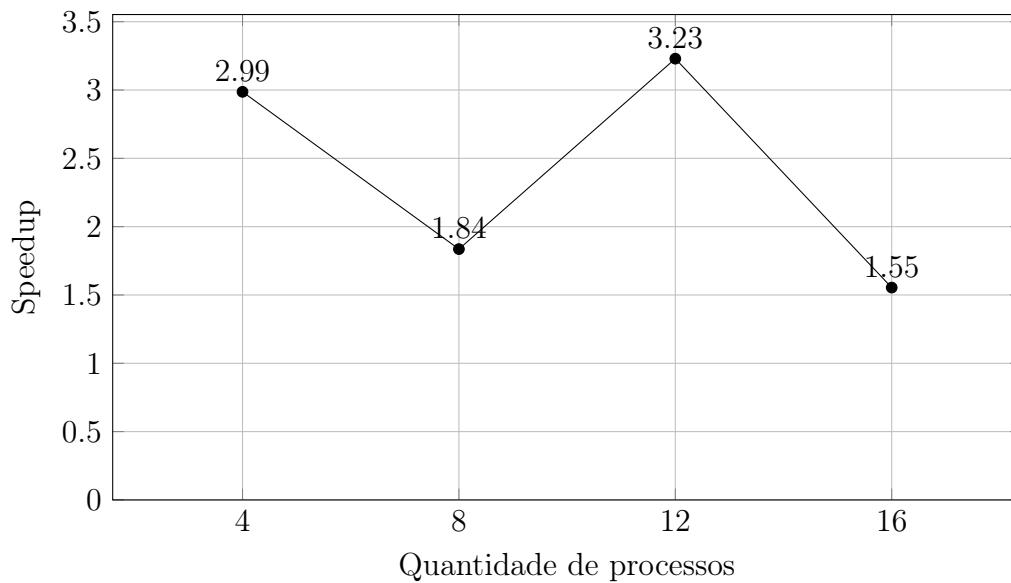


Figura 3: Speedup obtido em relação à execução sequencial.

O maior speedup foi obtido com **12 processos**, alcançando aproximadamente **3.23**. Isso significa que essa execução foi cerca de três vezes mais rápida que a versão sequencial.

11 Eficiência

A eficiência indica o quanto os processos foram aproveitados durante a execução paralela. Ela foi calculada a partir do speedup, dividindo esse valor pela quantidade de processos utilizados:

$$E(p) = \frac{S(p)}{p}$$

Para apresentar o resultado em porcentagem, o valor foi multiplicado por 100:

$$E(p) = \frac{S(p)}{p} \times 100$$

Processos	Speedup	Eficiência
4	2.9869	74.67%
8	1.8358	22.95%
12	3.2295	26.91%
16	1.5542	9.71%

Tabela 5: Eficiência obtida nas execuções paralelas.

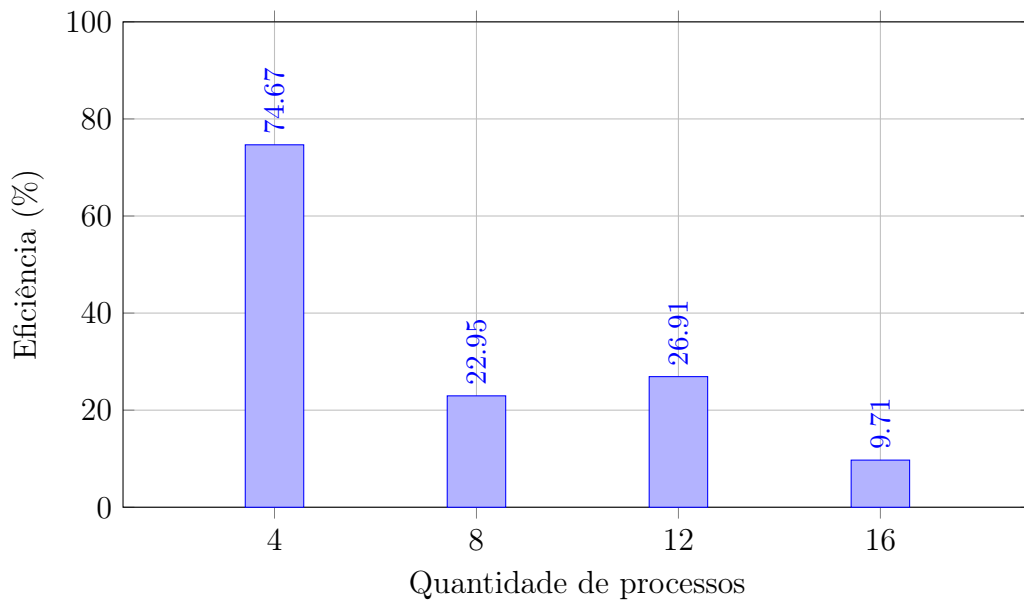


Figura 4: Eficiência das execuções paralelas.

A maior eficiência foi obtida com **4 processos**, alcançando **74.67%**. Isso indica que, nessa configuração, os processos foram melhor aproveitados em relação às demais execuções.

12 Lei de Amdahl

A fórmula do speedup segundo Amdahl é:

$$S(p) = \frac{1}{(1 - f) + \frac{f}{p}}$$

Nesta análise, em vez de assumir um valor fixo para f , a fração paralelizável foi estimada a partir do speedup medido em cada execução. Isolando f , tem-se:

$$f = \frac{1 - \frac{1}{S(p)}}{1 - \frac{1}{p}}$$

A Tabela 6 apresenta a estimativa da fração paralelizável para cada quantidade de processos.

Processos	Speedup medido	Fração paralelizável estimada
4	2.9869	88.69%
8	1.8358	52.03%
12	3.2295	75.31%
16	1.5542	38.04%

Tabela 6: Fração paralelizável estimada a partir da Lei de Amdahl.

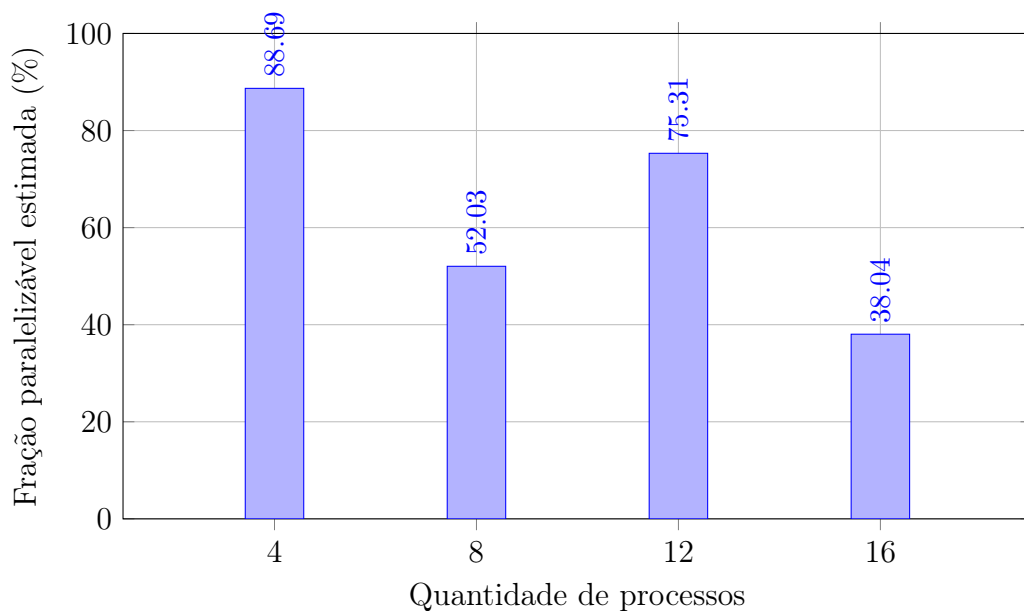


Figura 5: Estimativa da fração paralelizável efetiva pela Lei de Amdahl.

Os valores estimados de f variaram entre as execuções. A maior fração paralelizável

estimada ocorreu com **4 processos**, chegando a **88.69%**. Já a menor ocorreu com **16 processos**, com **38.04%**.

13 Logs de execução

```
rgm47538@l1m03u24:/home/local/rgm47538/mpi$ make run4
unset DISPLAY; mpirun --hostfile hosts_mpi_4 --mca btl_tcp_if_include eth0 --mca oob_tcp_if_include eth0 -np 4 /home/local/rgm47538/mpi/par /tmp/graph.bin
[rank 0/4] rodando em l1m03u24, recebeu 200000000 arestas locais
[rank 1/4] rodando em l1m11u24, recebeu 200000000 arestas locais
[rank 2/4] rodando em l1m13u24, recebeu 200000000 arestas locais
[rank 3/4] rodando em l1m12u24, recebeu 200000000 arestas locais
Peso total: 75117.848736822314
Tempo total: 150.940320 segundos
Arquivo gerado: saida_par.txt
```

Figura 6: Log da execução paralela com 4 processos.

```
rgm47538@l1m03u24:/home/local/rgm47538/mpi$ make run8
unset DISPLAY; mpirun --hostfile hosts_mpi_8 --mca btl_tcp_if_include eth0 --mca oob_tcp_if_include eth0 -np 8 /home/local/rgm47538/mpi/par /tmp/graph.bin
[rank 1/8] rodando em l1m11u24, recebeu 100000000 arestas locais
[rank 6/8] rodando em l1m20u24, recebeu 100000000 arestas locais
[rank 4/8] rodando em l1m14u24, recebeu 100000000 arestas locais
[rank 5/8] rodando em l1m19u24, recebeu 100000000 arestas locais
[rank 0/8] rodando em l1m03u24, recebeu 100000000 arestas locais
[rank 7/8] rodando em l1m21u24, recebeu 100000000 arestas locais
[rank 2/8] rodando em l1m12u24, recebeu 100000000 arestas locais
[rank 3/8] rodando em l1m13u24, recebeu 100000000 arestas locais
Peso total: 75117.848736822314
Tempo total: 245.580198 segundos
Arquivo gerado: saida_par.txt
```

Figura 7: Log da execução paralela com 8 processos.

```
rgm47538@l1m03u24:/home/local/rgm47538/mpi$ make run12
unset DISPLAY; mpirun --hostfile hosts_mpi_12 --mca btl_tcp_if_include eth0 --mca oob_tcp_if_include eth0 -np 12 /home/local/rgm47538/mpi/par /tmp/graph.bin
[rank 6/12] rodando em l1m20u24, recebeu 66666666 arestas locais
[rank 11/12] rodando em l1m29u24, recebeu 66666667 arestas locais
[rank 10/12] rodando em l1m28u24, recebeu 66666667 arestas locais
[rank 5/12] rodando em l1m19u24, recebeu 66666667 arestas locais
[rank 1/12] rodando em l1m11u24, recebeu 66666667 arestas locais
[rank 4/12] rodando em l1m14u24, recebeu 66666667 arestas locais
[rank 7/12] rodando em l1m21u24, recebeu 66666667 arestas locais
[rank 2/12] rodando em l1m12u24, recebeu 66666667 arestas locais
[rank 3/12] rodando em l1m13u24, recebeu 66666666 arestas locais
[rank 8/12] rodando em l1m22u24, recebeu 66666667 arestas locais
[rank 9/12] rodando em l1m27u24, recebeu 66666666 arestas locais
[rank 0/12] rodando em l1m03u24, recebeu 66666666 arestas locais
Peso total: 75117.848736822314
Tempo total: 139.602443 segundos
Arquivo gerado: saida_par.txt
```

Figura 8: Log da execução paralela com 12 processos.

```
rgm47538@l1m03u24:/home/local/rgm47538/mpi$ make run16
unset DISPLAY; mpirun --hostfile hosts_mpi_16 --mca btl_tcp_if_include eth0 --mca oob_tcp_if_include eth0 -np 16 /home/local/rgm47538/mpi/par /tmp/graph.bin
[rank 12/16] rodando em l1m31u24, recebeu 50000000 arestas locais
[rank 10/16] rodando em l1m28u24, recebeu 50000000 arestas locais
[rank 15/16] rodando em l1m15u24, recebeu 50000000 arestas locais
[rank 11/16] rodando em l1m29u24, recebeu 50000000 arestas locais
[rank 6/16] rodando em l1m20u24, recebeu 50000000 arestas locais
[rank 5/16] rodando em l1m19u24, recebeu 50000000 arestas locais
[rank 13/16] rodando em l1m23u24, recebeu 50000000 arestas locais
[rank 1/16] rodando em l1m11u24, recebeu 50000000 arestas locais
[rank 14/16] rodando em l1m05u24, recebeu 50000000 arestas locais
[rank 4/16] rodando em l1m14u24, recebeu 50000000 arestas locais
[rank 0/16] rodando em l1m03u24, recebeu 50000000 arestas locais
[rank 9/16] rodando em l1m27u24, recebeu 50000000 arestas locais
[rank 7/16] rodando em l1m21u24, recebeu 50000000 arestas locais
[rank 2/16] rodando em l1m12u24, recebeu 50000000 arestas locais
[rank 3/16] rodando em l1m13u24, recebeu 50000000 arestas locais
[rank 8/16] rodando em l1m22u24, recebeu 50000000 arestas locais
Peso total: 75117.848736822314
Tempo total: 290.081397 segundos
Arquivo gerado: saida_par.txt
```

Figura 9: Log da execução paralela com 16 processos.

14 Análise dos resultados

Os testes mostraram que a versão paralela produziu o mesmo peso total da AGM que a versão sequencial, igual a **75117.848736822314**. Isso indica que a paralelização manteve a corretude do algoritmo.

Em relação ao tempo de execução, todas as versões paralelas foram mais rápidas que a versão sequencial. O melhor tempo foi obtido com **12 processos**, com **139.602443 segundos**, enquanto a versão sequencial levou **450.847467 segundos**.

Apesar disso, o desempenho não cresceu de forma linear com o aumento da quantidade de processos. A execução com 4 processos teve boa eficiência, enquanto as execuções com 8 e 16 processos apresentaram tempos maiores. Isso mostra que, além da divisão das arestas, fatores como comunicação MPI, sincronização entre processos, distribuição inicial pelo rank 0 e diferenças entre as máquinas influenciaram os resultados.

15 Conclusão e Referências Bibliográficas

Neste trabalho foi implementada uma solução sequencial e uma solução paralela para o problema da Árvore Geradora Mínima. As duas versões utilizaram o algoritmo de Boruvka em conjunto com a estrutura Union-Find, permitindo controlar os componentes do grafo e evitar a formação de ciclos durante a construção da AGM.

A versão paralela foi desenvolvida com MPI, distribuindo as arestas entre diferentes processos. Cada rank processou sua parte local do grafo, enquanto o rank 0 ficou responsável por combinar as menores arestas encontradas e sincronizar as uniões entre os processos.

Dessa forma, conclui-se que a paralelização foi eficiente para reduzir o tempo de execução, mas seu desempenho depende fortemente do ambiente utilizado e dos custos de comunicação envolvidos.

Referências

- [1] JÁJÁ, Joseph. *An Introduction to Parallel Algorithms*. Addison-Wesley, 1992.
- [2] ESCOLA SDUMONT. *[Minicurso] Introdução Programação MPI*. YouTube, 13 jan. 2021. Professor: Carla Osthoff (LNCC). Disponível em: <https://www.youtube.com/watch?v=QSNQ76Rf-Us>. Acesso em: 20 mai. 2026.
- [3] PESQUISA OPERACIONAL – EXERCÍCIOS E APLICAÇÕES. *Problema da Mínima Arborescência (Árvore Geradora Mínima) e algoritmo de Boruvka*. YouTube, [s.d.]. Disponível em: <https://www.youtube.com/watch?v=rF7H2EwELPI>. Acesso em: 26 mai. 2026.
- [4] UEMS – Universidade Estadual de Mato Grosso do Sul. *Programação Paralela e Distribuída*. Página da disciplina, 2026. Disponível em: <https://www.comp.uems.br/~rubens/ppd.html>.